

Continuous interacting ant colony algorithm based on dense heterarchy

J. Dréo, P. Siarry*

*Laboratoire d'Étude et de Recherche en Instrumentation Signaux et Systèmes (L.E.R.I.S.S.),
Université de Paris XII Val-de-Marne, 61 Avenue du Général de Gaulle, 94010 Créteil, France*

Received 2 August 2002; received in revised form 18 June 2003; accepted 16 July 2003

Available online 20 February 2004

Abstract

Ant colony algorithms are a class of metaheuristics which are inspired from the behavior of real ants. The original idea consisted in simulating the stigmergic communication, therefore these algorithms are considered as a form of adaptive memory programming. A new formalization is proposed for the design of ant colony algorithms, introducing the biological notions of heterarchy and communication channels. We are interested in the way ant colonies handle the information. According to these issues, a heterarchical algorithm called “Continuous Interacting Ant Colony” (CIAC) is designed for the optimization of multim minima continuous functions. CIAC uses two communication channels showing the properties of trail and direct communications. CIAC presents interesting emergent properties as it was shown through some analytical test functions.

© 2003 Elsevier B.V. All rights reserved.

Keywords: Continuous optimization; Ant colony algorithm; Heterarchy

1. Introduction

Having recently given raise to a new metaheuristic method, the ant colony metaphor proved to be a successful approach to solve “difficult” optimization problems. Originally, there is the idea that a decentralized system showing a kind of *swarm intelligence* [2] can be very useful to solve problems, and particularly optimization problems. Nowadays there exists quite a lot of algorithms that can be considered to be more or less close to this idea, especially the *population based* algorithms, like Particle Swarm Optimization [8], Genetic Algorithms [12], Differential Evolution [16], Evolutionary Programming [9] and so on.

But, in our opinion, the best example of such systems is shown by the Ant Colony Optimization.

The first algorithm inspired from the ant colony functioning is the “ant system” [6], which has been applied to many combinatorial problems. Until now, there are few adaptations of such algorithms to continuous optimization problems. The first algorithm designed for continuous function optimization was CACO (for *Continuous Ant Colony Optimization*) [1] which comprises two levels: global and local. CACO uses the ant colony framework to perform local searches, whereas global search is handled by a genetic algorithm. Indeed, the “global” ants perform a simple evaluation of some regions defined in the search space, in order to update the regions fitness. The creation of some new regions is handled by a process very similar to a genetic algorithm, using common operators that are assimilated by the authors to some real

* Corresponding author. Tel.: +33-1-4517-1567;
fax: +33-1-4517-1492.

E-mail addresses: dreo@univ-paris12.fr (J. Dréo),
siarry@univ-paris12.fr (P. Siarry).

ant colonies behavior like “random walk” (playing the part of crossovers and mutations). The local level is handled by ants that explore more systematically the regions with a simple descending behavior, in order to move regions closer to the optimum. The algorithm sends some local ants on regions, these ants lay down some pheromonal spots when they find an improvement of the objective function and the spots are attractive for all the ants of the colony. This process is close to the original metaphor, but unfortunately, the use of two different processes inside the CACO algorithm leads to a delicate setting of parameters.

All these algorithms use a particular trait of real ants behavior: the pheromonal trail laying (Fig. 1a). Indeed, ant colonies are often viewed as distributed systems capable of solving complex problems by the way of *stigmergy*, which is a form of indirect communication mediated by modifications of environment. But the trail-laying behavior is also a part of the *recruitment* process, as the recruitment is defined by biologists as “a special form of assembly in which members of a society are directed to some point in space where work is required” [13, p. 642]. According to this definition, another optimization method has been developed for continuous optimization. The API algorithm is inspired by primitive ants behavior [15]. API uses a “tandem-running” which involves two ants and leads to gather the individuals on a same hunting site. The authors use this particular recruitment to make the population proceed towards the optimum, by selecting the best point among those evaluated by the ants. This procedure seems to be very similar to an enhanced random search as this algorithm makes a poor use of mem-

ory that generally characterizes ant colony systems [17].

Our point of view is that ant colony metaphor can be defined as a model using stigmergy or, more widely, as a recruitment process. According to this idea, pheromonal trail laying may not be the only way to comprehend the ant colony metaphor for optimization problems. Our recent research on modeling ants behavior [7] has shown that it is possible to start a recruitment sequence without taking pheromonal trails into account. In this model, the stigmergic processes are deliberately ignored and we focus on the inter-individual relationships. The model tried to reproduce the flow of ants exiting the nest after the entry of a scout who has discovered a new food source. To differentiate this process from the stigmergic recruitment, we called it *mobilization* (Fig. 1b). The mobilization is also a form of recruitment process: a scout comes back into the nest after having located a food source and induces the outing of several ants. We have shown that, through a simple modelization of trophallaxies (an exchange of liquid food between two individuals), a colony of ants can solve problems, like adapting the amplitude and the speed of the exiting flow to the scout's state. The main mechanisms involved in the response are the ants state distribution and the propagation of the mobilization information. Such a model suggests that the importance of inter-individuals communication may be underestimated, and that including it into an ant colony system may improve performances by accelerating the diffusion of information.

We were interested by these various ways of approaching the same idea and we thought that it would

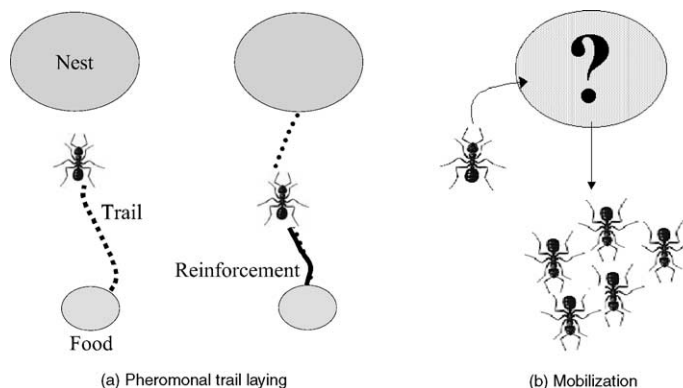


Fig. 1. Two views of the recruitment process.

be useful to gather them in a single formalism. Indeed, there is another interesting way of approaching the social insects behavior: the notion of *dense heterarchy*, developed in Section 2. As explained in Section 2.1, this is originally a description of the way the ant colony works from the point of view of communication, but we propose in Section 2.2 a simple formalization in order to use this notion to design an optimization algorithm. An *heterarchical algorithm* takes advantage from a flow of information passing through a population of agents. These pieces of information are exchanged using *communication channels* and permit that a form of description of the objective function emerges from the system. The new heterarchical algorithm that we have implemented is called *CIAC*, for *Continuous Interacting Ant Colony*. This paper comprises six more sections. In Section 2, we describe the notion of dense heterarchy. The communication channels used in CIAC are discussed in Section 3. Section 4 is devoted to the presentation of the CIAC algorithm. Then we discuss in Section 5 the tuning of CIAC parameters. In Section 6, we present some experimental results. Conclusion makes up the last section.

2. The notion of dense heterarchy

2.1. A biological definition

This notion was first introduced by Wilson in 1988 to describe the information flow inside an ant colony [18]:

“[An] ant colony is a special kind of hierarchy, which can usefully be called a heterarchy. This means that the properties of the higher levels affect the lower levels to some degree, but induced activity in the lower units feeds back to influence the higher levels”.

In this idea, the two communication channels that we have evoked in the introduction are present, the trail channel as well as the direct communication between individuals. Indeed, these two channels are relatively important for the ants and thus are good examples to comprehend the notion of heterarchy. One of the important issues here is that informations *flow* through the colony, each of the ants can communicate with any other (which makes the heterarchy being *dense*).

This process constructs a kind of densely connected network which is not set up in a *hierarchical* but in a *heterarchical* manner. A good metaphor consists in emphasizing the opposite behavior of an ant colony in comparison with that of a human army, where a general gives orders to a colonel who commands a group of lieutenants and so on . . . In the ant colony, contrary to a popular belief, the “queen” does not tell the workers what to do, but is only a part of the network (cf. Fig. 2). Such a system has emergent properties, indeed if each agent operates with elementary rules and a limited accuracy, the whole population organized in heterarchy with mass communication can show an emergent pattern. These findings are well known in the study of self-organization [3].

To summarize, the notion of dense heterarchy describes the way the ant colony handles the pieces

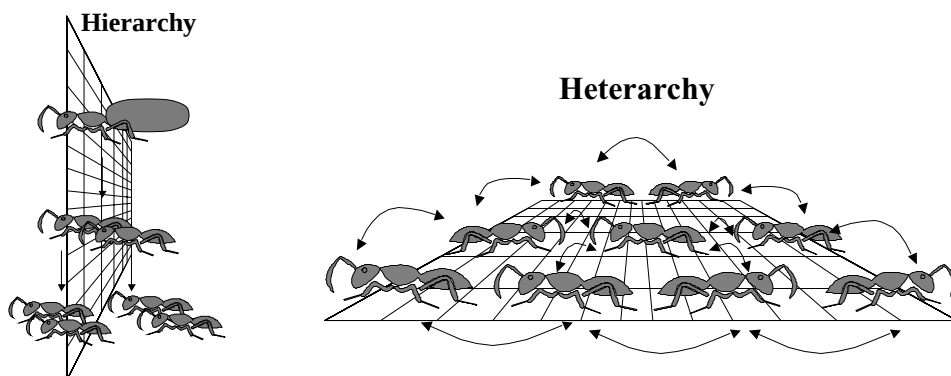


Fig. 2. In a dense heterarchy, unlike hierarchy, any individual can communicate with any other one.

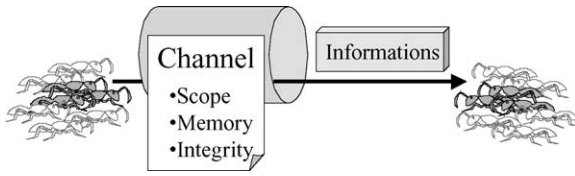


Fig. 3. The informations pass from a part of the population to another one through a communication channel that has some specific properties.

of information that it receives from the environment. Each ant can communicate with any other at any time and the pieces of information flow inside the colony through many communication channels.

2.2. Heterarchical algorithms

We propose here a simple formalization in order to apply the notion of dense heterarchy to an optimization problem. The main concept for implementing such an *heterarchical algorithm* is the idea of *communication channel* (cf. Fig. 3). A communication channel may be, to take an example in the “ant metaphor”, the lay of pheromonal trail. These channels transmit an information, here the localization of a food source, and have some properties, like stigmergy and memory.

For example, some basic properties of the channels are listed below:

- (1) *Scope*: the way the information goes through the population. A sub-group of the population (from 1 to n agents) can exchange informations with another group of agents.
- (2) *Memory*: the way the information persists in the system. The information can be stored during some period of time or be transitory.
- (3) *Integrity*: the way the information is evolving in the system. The information can be modified, by one or more agents, by an external process, or not.

These properties can be combined in the same channel, so that a large variety of different channels can be built.

The information transmitted during the communication can take many forms, from a simple value to a complex “object”, therefore it is difficult to describe some particular classes. The more intuitive ones are, for example, the vector coordinates of a point and the value of the objective function at this location.

As an example, let's take a look at the properties of the “trail-laying” channel. Basically, the *scope* is potentially the whole population, as each ant can perceive the trail pheromone. There is also a form of *memory*, as this is a stigmergic process, the trail persists in the environment during a certain period of time. Finally, the *integrity* of the channel permits that the informations are damaged by the time, as the pheromones evaporate.

3. The CIAC communication channels

The design of the CIAC algorithm is quite simple, under the look of the heterarchical concept. We have implemented three versions, each one defined by the type of communication channel it uses. As we will see in next sections, this is very useful to comprehend the way the algorithm works.

3.1. The trail channel

The first version of CIAC algorithm is trying to be as close as possible to a continuous version of the original ACO (Ant Colony Optimization) algorithm [6] which was designed for combinatorial problems. Due to this assumption, it has somewhat the same design as the local search part of the CACO algorithm [1], which was also inspired by the first Ant System.

This implementation uses only one communication channel which is inspired from the trail-laying behavior of ants. Here, each ant can deposit a certain amount of *pheromone* as a spot on the search space, proportionally to the improvement of the objective function she founds on her way. These pheromonal spots can be perceived by all the members of the population, and diffuse into the environment. The ants are attracted by each spot according to its distance and the amount of pheromone it contains (cf. (1)). The agents are moving towards the gravity center G_j of the pheromonal spots cloud. The position of the gravity center depends on the interest ω_{ij} of the j th ant for the i th spot:

$$G_j = \sum_{i=1}^n \left(\frac{x_i \cdot \omega_{ij}}{\sum_{i=1}^n (\omega_{ij})} \right) \quad \text{with} \quad \omega_{ij} = \frac{\bar{\delta}}{2} \cdot e^{(-\theta_i \cdot \delta_{ij})}. \quad (1)$$

Here, n is the number of spots and x_i the position of the i th spot. The variables involved in the calculation

of the interest ω_{ij} are: $\bar{\delta}$ the mean distance between two agents in the population, θ_i the amount of pheromone laid on the spot and δ_{ij} the distance between the j th ant and the i th spot. It is important to notice that at this point each ant does not go directly on the gravity center. Indeed, each artificial ant has a “range” parameter (noted ϕ_j) that is normally distributed over the population. Each artificial ant draws a random distance, under the limit of her range parameter, and then jumps of this length in the direction of her weighted gravity center, some noise modifying the final position.

To summarize this behavior under the vision of the heterarchical concept, this trail communication channel shows the following properties, underlined in [Section 2.2](#):

- (1) *Scope*: when one ant lays a pheromonal spot, all the ants can subsequently perceive it.
- (2) *Memory*: the information persists in the system during a certain period of time, independently of the agents.
- (3) *Integrity*: the information is modified by time, to reproduce the pheromone evaporation.

There is some similarity between this algorithm and the “path-relinking” algorithm introduced by Glover and Laguna [10], as the ants are moving through a set of informative points.

3.2. Using the direct inter-individual communication

As we have said in [Section 1](#), some biological works have led us to take an interest out of the common vision of the “stigmergic” ant colony optimization. Indeed, we have implemented another communication channel which possesses the properties of the direct inter-individual interactions that can be observed in societies of some social insects as ants.

In concrete terms, each ant can send “messages” to another one, this means that the scope of this communication channel is from “peer to peer”. An ant receiving a message stores it in a stack with other incoming messages. In a second time, a message is read randomly in the stack. This process is also inspired from the works on communication problems in large multi-agent systems [11], more particularly on the implementation of parallel programs. Here, the information sent is the position of the sender—an artificial ant—and the value of the objective function.

The receiver compares the sender’s value with its own value and decides if it moves near the sender’s position. The final position is drawn randomly in a hyper-sphere which has the sender for center and the receiver’s range for radius. But if the receiver’s value is better than the sender’s one, then the receiver sends a message to another ant randomly chosen, and suppresses the read message. One can notice that the system needs to be “activated”, so that an important parameter is the number of messages initially set.

This communication channel shows the following properties:

- (1) *Scope*: when an ant sends a message, only one ant can perceive it.
- (2) *Memory*: the information persists in the system during a certain period of time, under the form of ant’s memory.
- (3) *Integrity*: the informations stored are static.

3.3. The final algorithm

Finally, the two simple algorithms previously described ([Sections 3.1 and 3.2](#)) are designed to be quite different. We have approached in [Section 2.1](#) the notion of self-organization where a system made of simple units can show emergent properties at a high level. According to this idea, we have implemented in the same system the two simple communication channels, in order to study if they work in synergy or not. This combination is quite simple to do, as the communication channels have no concurrent process.

4. The CIAC algorithm

The algorithm comprises three main steps ([Fig. 4](#)). In the first one, the parameters are set up, notably the ants ranges are distributed over the agents population and they are put randomly within the search space. Then the algorithm starts and the ants move until a stopping criterion is reached: CIAC is stopped if the difference between two consecutive best points is lower than 10^{-3} during $(\eta \cdot 10)$ evaluations (see below for the definition of η) or if a maximum number of objective function evaluations has been reached. Ants are moving according to their own perception of the system, which is handled by the communication channels.

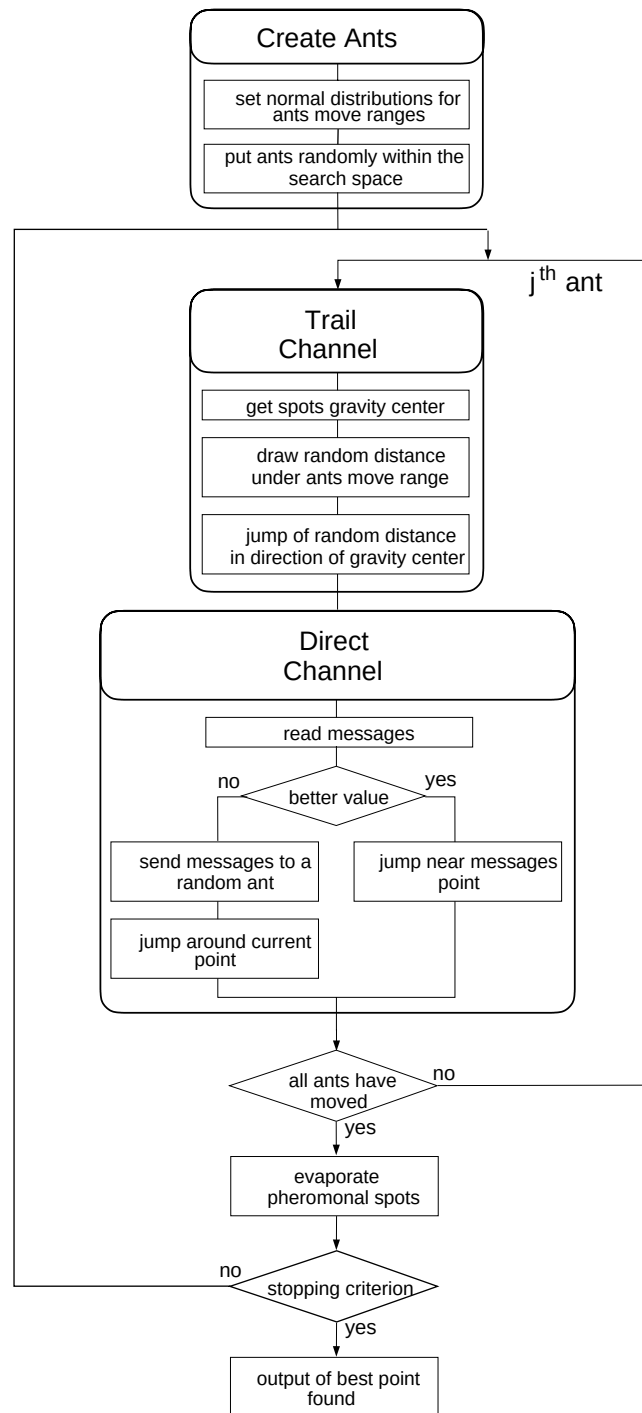


Fig. 4. The basic structure of CIAC.

There are four parameters which need to be set. These parameters have the following functions:

- (1) $\eta \in [0, +\infty[$: the number of agents in the system.
- (2) $\sigma \in [0, 1]$: a percentage of the search space amplitude. Used to define the standard deviation of the normal distribution of ants moves ranges.
- (3) $\rho \in [0, 1]$: defines the pheromonal spots persistence.
- (4) $\mu \in [0, +\infty[$: sets the initial number of messages.

The setting of those four parameters is discussed in [Section 5](#).

In addition, there are some other parameters which are included in the algorithm or deduced from the chosen parameters:

- (1) The distribution of the ants starting points. The ants are distributed randomly within the search space.
- (2) ε : under this value of pheromone quantity, a spot disappears. It is set to the minimum value handled by the programming language (here 10^{-14}).
- (3) ς : the noise amplitude added to the ant position when the ant moves according to the trail channel. This is a percentage of the ants move range. It is set according to (2) (cf. [Section 3.1](#) for variables names):

$$\begin{cases} \frac{\bar{\delta}}{\phi_j} & \text{if } \bar{\delta} \leq \phi_j, \\ 1 & \text{else.} \end{cases} \quad (2)$$

5. Tuning of CIAC parameters

In this section we assume that—as in many real-world problems—the problem is a black box and that only the dimension is available, not any other information like the number of local optimums or the look of the objective function.

The number of ants η is not a critical parameter as it does not influence critically the overall convergence of the algorithm. With only 10 ants, the algorithm performs well but the need for more ants increases with the dimension of the objective function. In fact, a set of 100 ants permits the algorithm to be flexible enough over all the functions as well of 2 than of 100 dimensions. On the other hand, the ranges distribution ratio

σ influences the efficiency of the algorithm. Indeed, it defines the way the search space will be explored: a too small value will lead to an insufficiently explored search space and a too high value will lead to random movements. But with experiments, it is easy to show that a relatively high value of σ is recommended (typically: 0.9). The two other parameters are interesting because they are specific to the two communication channels implemented. The first one, the persistence of the pheromonal spots ρ , is quite sensitive, because a high value can easily lead the ants to be trapped into a local optimum. Here also, the experience tells us to use a small value (typically: 0.1) to avoid this issue on the largest set of functions. The last parameter is μ , the initial messages number, that is specific to the direct inter-individual communication channel. It is also relatively critical because a too high value will induce a reduction of the trail channel influence.

After several trials using a set of 13 classical analytical test functions of 2–100 variables, we have designed some rules to automatically set up the parameters so that only two parameters need to be set up manually. Default values for the parameters that are not automatically set were fixed as follows: $\rho = 0.1$ and $\sigma = 0.9$. These settings represent a compromise over all the test functions used and thus are not necessarily the best ones for a particular function. Moreover, we have decided to put the accent on the efficiency of the algorithm (i.e. the capability to find an approximative value of the global optimum), more than on the rapidity (i.e. the number of evaluations of the objective function) and the accuracy (i.e. the final distance to the exact global optimum). The rules of automatic setting are mainly dependent of the number of dimensions of the objective function, since it proved to be the most effective approach. The number of ants is set according to (3):

$$\eta = \eta_{\max}(1 - e^{(-d/p)}) + \eta_0 \quad (3)$$

with

- d the number of dimensions of the objective function,
- $\eta_{\max} = 1000$ the maximum number of ants,
- $\eta_0 = 5$ the number of agents for zero dimensions,
- $p = 10$ the relative importance of the ants number,
- $\mu = (\eta \cdot 2/3)$ the initial number of messages.

This way of setting the ants number permits to fit the population size to the problem size with a moderate number of evaluations.

To summarize, the two most sensitive parameters are those which are specific to the communication channels implemented in the algorithm, and generally the algorithm is efficient when there is a “balance” between ρ and μ . The automatic settings proposed above achieved such a balance.

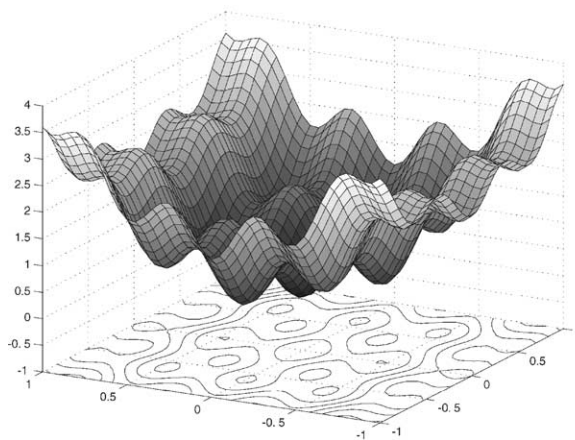
6. Experimental results

6.1. CIAC particular behavior

To illustrate the behavior of CIAC, we have applied it on the B_2 function (see (4)), which has two dimensions and some:

$$B_2(x_1, x_2) = x_1^2 + 2x_2^2 - 0.3 \cos(3\pi x_1) - 0.4 \cos(4\pi x_2) + 0.7. \quad (4)$$

The global optimum is $B_2 = 0$ at $(0, 0)$. We use a search interval of $[-50, 100]$. We also use only 10 ants to make the figures more readable, all the other parameters values are those described above.

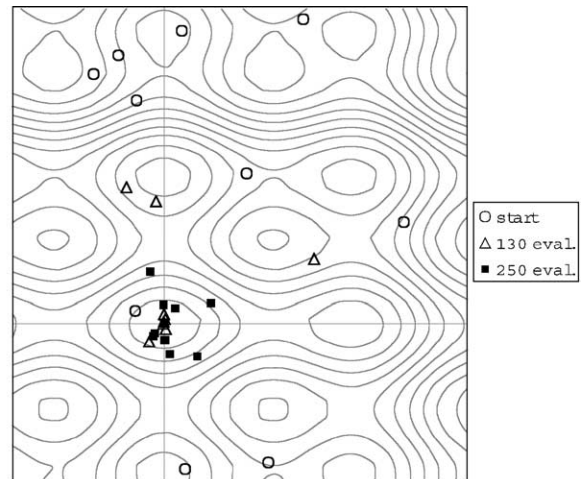


(a) the B_2 function near the global optimum

The two channels play complementary parts. Indeed, the direct channel leads to a form of intensification, because it gives more importance to the best points without taking into account the previously encountered regions. On the contrary, the trail channel—with its memory property—permits to perform a kind of diversification, by taking into account the previously evaluated points. Fig. 5b shows how the algorithm works with both channels. Globally, the ants gather at the global optimum. However some of them are kept during some iterations near local optima by some pheromonal spots still persisting within the search space, afterwards evaporation and direct communications prevent them from being trapped.

The way the trail channel permits a form of diversification is pointed out with more difficulties. Fig. 6 shows CIAC using only this communication channel. These four consecutive snapshots are taken after 101, 102, 103 and 104 evaluations of the objective function. The ants are moving around a gravity center that is the global optimum, but continue to explore the search space, as they are attracted by local minima.

One interesting issue is the way the two channels works in synergy. Indeed, on the simple B_2 function, the direct channel seems to be more appropriate as it allows CIAC to converge more rapidly (Fig. 7a). But



(b) CIAC optimizing B_2

Fig. 5. CIAC optimizing B_2 function with the two channels, three steps are shown: at the beginning, after 130 and 250 evaluations of the objective function.

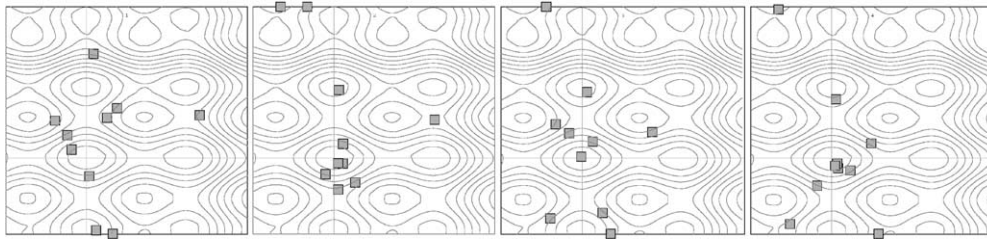


Fig. 6. CIAC optimizing B_2 function using only the trail channel, four consecutive steps are shown.

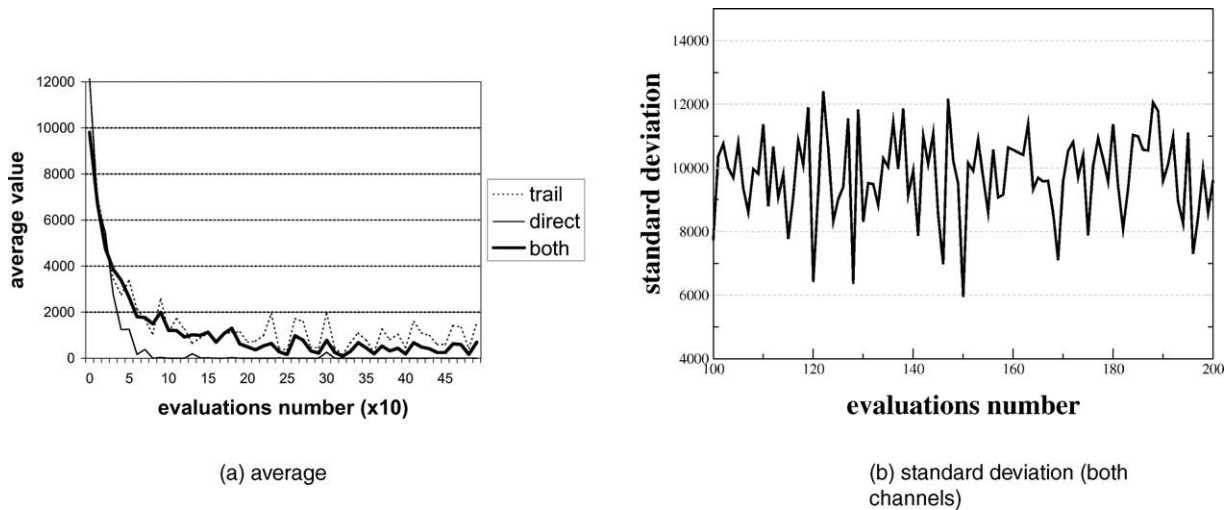


Fig. 7. Average and standard deviation of the objective function values in the population at different steps.

as we will see in Section 6.2, CIAC is subsequently trapped into a local optimum. If we take a look at the variation of the standard deviation for the algorithm using the two communication channels (Fig. 7b), we notice that there is some *oscillations*. This behavior means that the population tends to gather near a value at one time and then tends to disperse. In other words, there is an alternation of short intensification phases (low deviation) and diversification phases (high deviation).

This behavior of the CIAC algorithm cannot be observed when only one of the two channels is used (as shown in Fig. 8). Indeed, with only one channel, the standard deviation decreases and the amplitude of the oscillations is smaller: the standard deviation of the evolution of both channels (an index of the dispersion) is $\sigma_{\text{both}} = 1581$. For the trail channel alone $\sigma_{\text{trail}} = 1100$ and for the direct channel alone $\sigma_{\text{direct}} = 911$.

Thus CIAC regulates itself the way the two channels are working, until a stable state is found. We call this behavior an emergent pattern because there is no centralized control of the system and the ants are

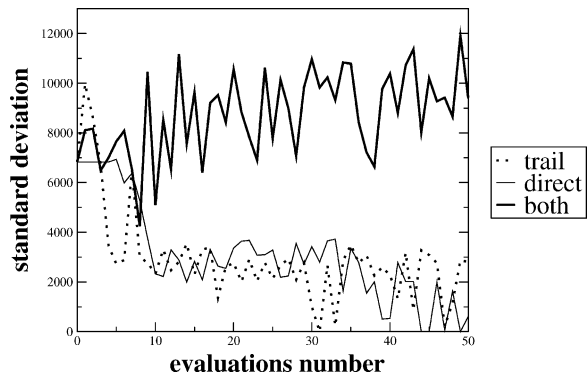


Fig. 8. Standard deviation of the objective function values in the population at different steps for the different channels.

locally interacting without any access to the whole information flowing in the system. So that observing a global pattern (here the specific behavior of the system when using both channels) is not planned by advance in any form. Other examples of self-organized systems are described in [2].

6.2. Test results

We have tested the CIAC algorithm over a set of analytical test functions found in the literature. In order to compare CIAC with other continuous optimization algorithms (see in Appendix A for the list of the algorithms), we have chosen a set of 13 test functions partly in the literature devoted to continuous ant colonies algorithms [1,14,15], and partly in papers dealing with other continuous optimization algorithms [4,5,16]. These functions are listed in Appendix B. It can be pointed out that CIAC can handle as well functions to be minimized as functions to be maximized.

To avoid any problem due to the choice of an initial population, we performed each test 100 times with a different random seed at each test. The values of the objective function evaluation number (evals) and of the average error (err) are obtained through averaging the results over all the tests (the standard deviations are given in parenthesis). A test is considered to be successful (thus contributing to “% ok”) if (5) held:

$$|f^\alpha - f^*| < \epsilon_1 \cdot f^* + \epsilon_2 \quad (5)$$

with

- f^* the global optimum of the objective function,
- f^α the optimum found by the algorithm,

- ϵ_1 and ϵ_2 accuracy parameters: in all tests discussed in that paper, $\epsilon_1 = \epsilon_2 = 10^{-4}$.

First, we have tested the different versions of CIAC, implementing only one of the two channels or both. The results for the three different algorithms are shown in Table 1.

One can notice, in term of efficiency, that the algorithm implementing only the direct channel cannot find the global optimum and is easily trapped into local minima. On the contrary, the trail channel permits to avoid this behavior and leads to a better efficiency. The combination of both channels permits to find the global optimum more often than when using a single channel. But this improvement has a cost and the number of evaluations necessary to reach the stopping criterion is greater.

In order to compare CIAC with two competing continuous ant colony algorithms, we have performed some tests with the same test functions than those used in the related articles. Results are presented in Table 2. The value given in square brackets describes a test with a fixed number of evaluations, without any other stopping criterion and empty cells stand when values are not given in the literature [1,15].

Due to some lack of information in the literature, the comparison cannot be exhaustive, notably the efficiency and the rapidity of API are not specified in the related paper and some pieces of information are lacking for CACO. CIAC works worse than CACO in most cases, and seems to be as good as API. It can be noticed that the number of evaluations of CIAC is sometimes a little higher than those of CACO or API, probably due to the better accuracy of CIAC and the

Table 1
Comparison of CIAC versions over a subset of eight test functions

Func.	Direct			Trail			Both		
	% ok	evals	err	% ok	evals	err	% ok	evals	err
SM	1	39735 (104)	7.53 (2.9)	100	39899 (58)	1E–8 (2E–8)	100	50050 (76)	9E–10 (1E–11)
B ₂	11	5868 (5964)	1.95 (2.01)	76	5964 (51)	2E–3 (3E–3)	100	11827	2E–8 (1E–7)
R ₂	18	5794 (50)	16.90 (33.54)	88	5943 (54)	4E–3 (9E–3)	100	11797 (129)	3E–3 (3E–3)
R ₅	1	19594 (192)	295 (375)	75	19808 (103)	1E–2 (2E–2)	90	39546 (289)	8E–3 (7E–3)
GP	4	11561 (30)	30 (32)	54	11777 (103)	1.63 (2.59)	56	23391 (48)	1.51 (2.33)
Gr ₁₀	0	50000 (29)	1.1 (0.24)	5	50000 (33)	0.62 (0.14)	52	50121 (49)	0.05 (0.05)
MG	2	5826 (45)	0.71 (0.53)	12	5901 (53)	0.72 (0.74)	20	11751 (61)	0.34 (0.19)
S _{4,5}	1	19458 (86)	–1.96 (0.73)	48	19755 (154)	3.38 (2.74)	5	39311 (294)	6.34 (1.01)

Table 2
Results of CIAC against two competing continuous ant colony algorithms

Function	CACO			API			CIAC		
	% ok	evals	err	% ok	evals	err	% ok	evals	err
R ₂	100	6842	0.00		[10000]	0.00 (0.00)	100	11797	3E–3 (3E–3)
SM	100	22050			[10000]	0.00 (0.00)	100	50000	9E–10 (1E–11)
Gr ₅					[10000]	0.18 (0.04)	63	48402	0.01 (9E–3)
Gr ₁₀	100	50000	0.0				52	50121	0.05 (0.05)
GP	100	5330					56	23391	1.51 (2.33)
MG	100	1688					20	11751	0.34 (0.19)
St		[6000]	0.0				94	28201	1.96 (1.61)
B _{f1}		[200000]	544				0	50000	99521 (463)
B _{f2}		[200000]	877				0	50000	99635 (512)
B _{f3}		[200000]	54231				0	50000	99895 (86)

Table 3
Results of CIAC against three competing continuous optimization algorithms

Function	CGA			ECTS			DE		CIAC		
	% ok	evals	err	% ok	evals	err	% ok	evals	% ok	evals	err
St							100	1300	94	28201	1.96 (1.61)
Gr ₁₀							100	12804	52	50121	0.05 (0.05)
SM	100	750	0.0002	100	338	3E–8	100	392	100	50000	9E–10 (1E–11)
R ₂	100	960	0.004	100	480	0.02	100	615	100	11797	3E–3 (3E–3)
R ₅	100	3990	0.15	100	2142	0.08			90	39546	8E–3 (7E–3)
GP	100	410	0.001	100	231	2E–3			56	23391	1.51 (2.33)
S _{4,5}	76	610	0.14	75	825	0.01			5	39311	6.34 (1.01)

importance given to the efficiency in our tests. The Baluja's functions are a particular case: in two dimensions their plottings present no local minima but only a single global minimum looking as a constricted hole (see Fig. 10b–d in Remark). In such case, a simple descent algorithm should be more efficient.

Finally, we have tested CIAC using the same problems than those used in the literature devoted to various metaheuristics. Table 3 shows the results. For the DE method, the values of “err” are not available.

For all the functions, CIAC is far from the rapidity of other algorithms and is worse in term of efficiency. But CIAC is most times comparable in term of accuracy. These results are pointing out the inherent cost in CPU time (commonly counted in number of evaluations of the objective function) of the ant colony optimization algorithms, as was shown out in Table 2.

7. Conclusion

We have shown that the heterarchical concept can be interesting to design new ant colony algorithms, in particular aimed at the optimization of continuous multim minima functions. Such a biological concept was not exploited until now for the design of optimization algorithms, using only stigmergic processes. We have proposed to extend the ant colony metaphor to take into account several communication processes. CIAC is a heterarchical algorithm implementing two complementary communication channels. It shows interesting emergent properties, like a self-management of the relative influence of the two channels.

When comparing CIAC with competing algorithms, we pointed out that the defaults inherited from the ant colony algorithms concept are also present. The main problems are the cost in term of number of

evaluations of the objective function and the relatively bad efficiency. But CIAC also inherits the qualities of this concept, such as self-management of its global behavior and flexibility.

Appendix A. Competing algorithms

Competing continuous optimization algorithms used in the paper

Abbr.	Complete name	Type	Reference
ECTS	Enhanced Continuous Tabu Search	Tabu search	[5]
CGA	Continuous Genetic Algorithm	Genetic algorithm	[4]
DE	Differential Evolution	Population based	[16]
CACO	Continuous Ant Colony Algorithm	Ant colony	[1]
API	(from the ant <i>Pachycondyla apicalis</i>)	Ant colony	[15]
CIAC	Continuous Interacting Ant Colony	Ant colony	This paper

In its present form, we argue that CIAC can be fruitfully used for a global search for the most “promising” regions within the search space, but CIAC has to be hybridized with some local search classical algorithm for a faster and more accurate localization of the best solution.

The fact that ant colonies are not as good as other “classical” metaheuristics on common static test functions may not lead to the conclusion that the ant colony metaphor cannot be interesting for continuous optimization. Indeed if we take a look at both the advantages and the drawbacks of the ant colony metaphor, we can see that these algorithms bear medium performance when handling analytical test functions. Indeed, such systems are not known to have great performances in static (combinatorial) optimization problems but to they seem to be well suited to dynamical problems, due to their distributed and adaptive features [2].

We believe that two characteristics of CIAC are especially interesting. First, the use of the heterarchical concept through the implementation of communication channels making a link between search processes. Secondly, we point out the idea that ant colony algorithms are more efficient on dynamical problems, probably even on continuous spaces. These ideas should supply a framework to achieve simple and efficient optimization algorithms in a self-organized and flexible system, as needed in dynamical problems.

In view of these issues, our further work will try to take advantage of the CIAC potential to design a high-level heterarchical framework aimed at continuous dynamical optimization.

Appendix B. Test functions

Hereafter, the test functions used in the paper are described according to the following notations:

- *Dim.* the dimension of the function,
- $\vec{x}$ the variables vector,
- $\vec{x}^*$ the globally best solution (unspecified coordinates),
- $x_i \in [\dots]$ the search space range, for each variable,
- *Glob. Min.* or *Glob. Max.* the value and (if known) the position of the global optimum,
- *Nb. Loc. Min.* or *Nb. Loc. Max.* the number of local optima.

In Fig. 9 we show the plotting of six test functions to be minimized and in Fig. 10 we show the plotting of four functions to be maximized.

B.1. (B_2) B_2 (plotted in Fig. 9a)

- *Dim.*: 2
- $B_2(\vec{x}) = x_1^2 + 2x_2^2 - 0.3 \cos(3\pi x_1) - 0.4 \cos(4\pi x_2) + 0.7$
- $x_i \in [-100, 100]$
- *Glob. Min.*: $B_2(0, 0) = 0$

B.2. (MG) Martin & Gaddy (plotted in Fig. 9b)

- *Dim.*: 2
- $MG(\vec{x}) = (x_1 - x_2)^2 + ((x_1 + x_2 - 10)/3)^2$
- $x_i \in [-20, 20]$
- *Glob. Min.*: $MG(5, 5) = 0$

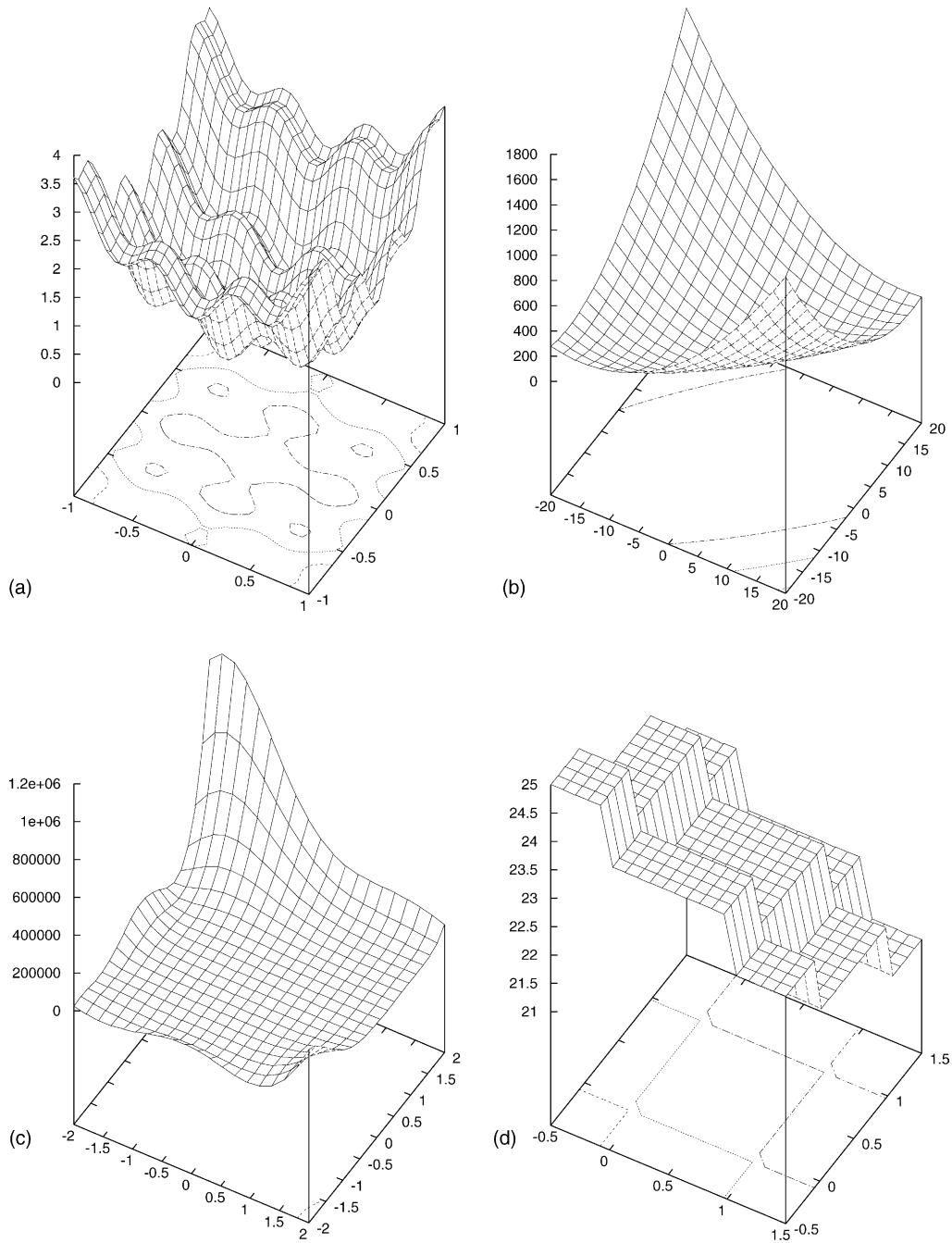


Fig. 9. Plottings of six test functions (to be minimized): (a) B_2 with $x_i \in [-1, 1]$, (b) MG with $x_i \in [-20, 20]$, (c) GP with $x_i \in [-2, 2]$ and (d) St with $x_i \in [-0.5, 1.5]$.

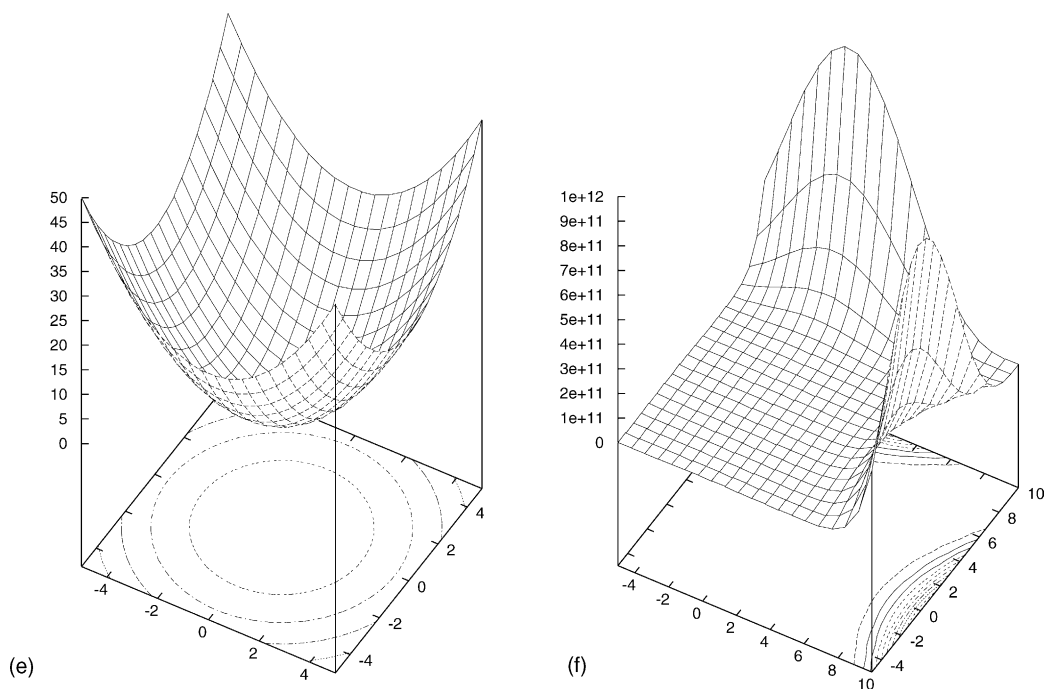


Fig. 9. (Continued).

B.3. (GP) Goldstein & Price (plotted in Fig. 9c)

- *Dim.*: 2
- $GP(\vec{x}) = (1 + (x_1 + x_2 + 1)^2 \cdot (19 - 14x_1 + 3x_1^2 - 14x_2 + 6x_1x_2 + 3x_2^2)) \cdot (30 + (2x_1 - 3x_2)^2 \cdot (18 - 32x_1 + 12x_1^2 + 48x_2 - 36x_1x_2 + 27x_2^2))$
- $x_i \in [-2, 2]$
- *Glob. Min.*: $GP(0, -1) = 3$

B.4. ($S_{4,n}$) Shekel

- *Dim.*: 4
- $S_{4,n}(\vec{x}) = -\sum_{i=1}^n (1/((x - a_i)^T(x - a_i) + c_i))$ with $x = (x_1, x_2, x_3, x_4)^T$; $a_i = (a_i^1, a_i^2, a_i^3, a_i^4)^T$ and $n \in \{1, 2, 3\}$
- $x_i \in [0, 10]$
- *Glob. Min.*: $S_{4,5}(\vec{x}^*) = -10.1532$; $S_{4,5}(\vec{x}^*) = -10.40294$; $S_{4,5}(\vec{x}^*) = -10.53641$
- *Nb. Loc. Min.*: $(n - 1)$

i	a_i^T				c_i
1	4	4	4	4	0.1
2	1	1	1	1	0.2
3	8	8	8	8	0.2
4	6	6	6	6	0.4
5	3	7	3	7	0.4
6	2	9	2	9	0.6
7	5	5	3	3	0.3
8	8	1	8	1	0.7
9	6	2	6	2	0.5
10	7	3.6	7	3.6	0.5

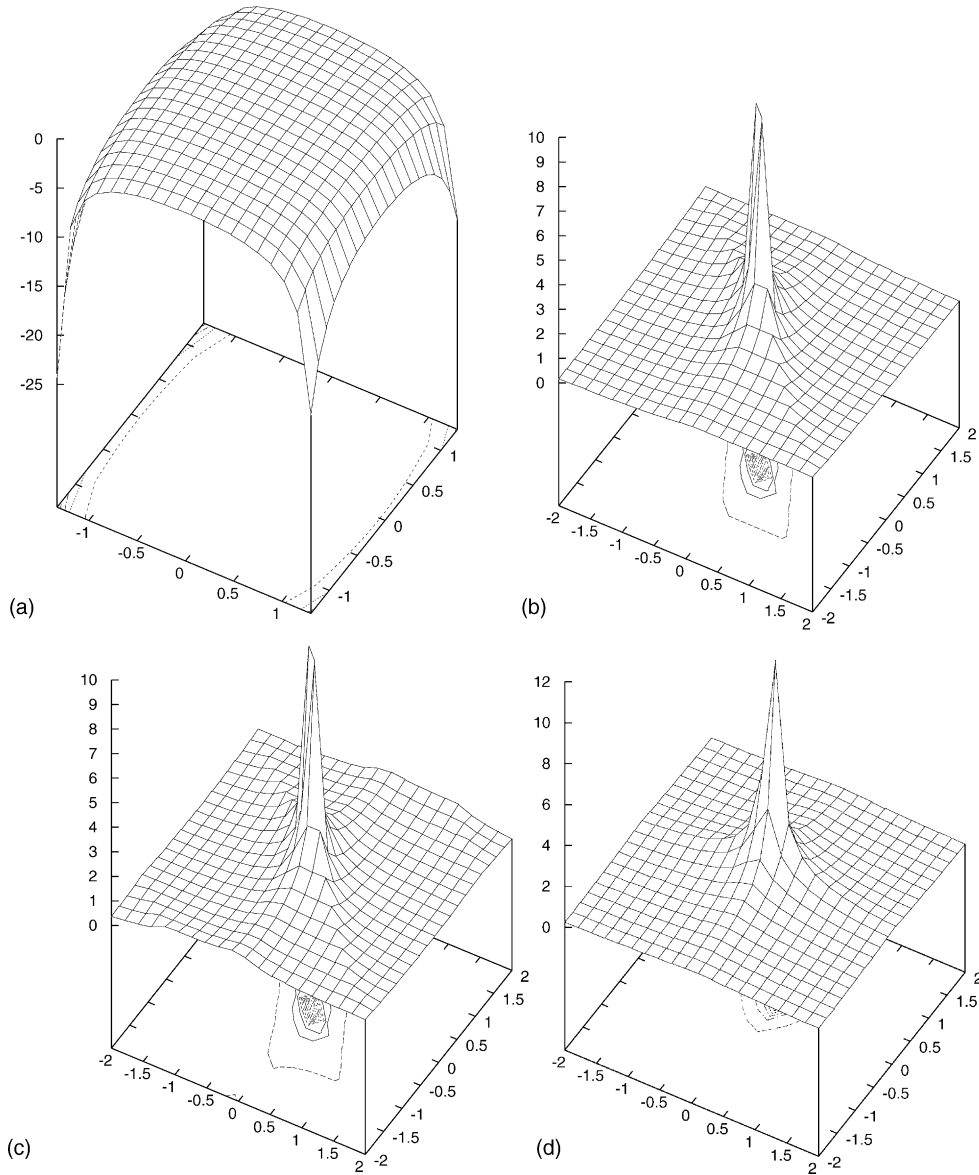


Fig. 10. Plottings of four test functions (to be maximized): (a) Gr_2 with $x_i \in [-2, 2]$, (b) B_{f_1} with $x_i \in [-2, 2]$, (c) B_{f_2} with $x_i \in [-2, 2]$ and (d) B_{f_3} with $x_i \in [-2, 2]$.

B.5. (St) Step (plotted in Fig. 9d)

- *Dim.*: 5
- $St(\vec{x}) = 25 - \sum_{i=1}^5 [x_i]^*$ with $[x_i]^*$ greatest integer $\leq x$
- $x_i \in [-5.12, 5.12]$
- *Glob. Max.*: $St(\vec{x}^*) = 55$

B.6. (SM) Sphere Model (plotted in Fig. 9e)

- *Dim.*: 6
- $SM(\vec{x}) = \sum_{i=1}^6 x_i^2$
- $x_i \in [-5.12, 5.12]$
- *Glob. Min.*: $SM(0, \dots) = 0$

B.7. (B_{f_1}) Baluja f1 (plotted in Fig. 10b)

- **Dim.:** 100
- $B_{f_1}(\vec{x}) = \left(\epsilon + \left| |y_1| + \sum_{i=1}^{100} |y_i| \right| \right)^{-1}$ with $y_1 = x_1$, $y_i = x_i + y_{i-1}$
- $x_i \in [-2.56, 2.56]$
- **Glob. Max.:** $B_{f_1}(0, \dots) = 100\,000$

B.8. (B_{f_2}) Baluja f2 (plotted in Fig. 10c)

- **Dim.:** 100
- $B_{f_2}(\vec{x}) = \left(\epsilon + \left| |y_1| + \sum_{i=1}^{100} |y_i| \right| \right)^{-1}$ with $y_1 = x_1$, $y_i = x_i + \sin(y_{i-1})$
- $x_i \in [-2.56, 2.56]$
- **Glob. Max.:** $B_{f_2}(0, \dots) = 100\,000$

B.9. (B_{f_3}) Baluja f3 (plotted in Fig. 10d)

- **Dim.:** 100
- $B_{f_3}(\vec{x}) = \left(\epsilon + \sum_{i=1}^{100} |0.024 \cdot (i+1) - x_i| \right)^{-1}$
- $x_i \in [-2.56, 2.56]$
- **Glob. Max.:** $B_{f_3}(0, \dots) = 100\,000$

B.10. (Gr_n) Griewangk (Gr_2 plotted in Fig. 10a)

Note: Gr_5 and Gr_{10} are used in the paper.

- **Dim.:** n
- $Gr(\vec{x}) = \left(0.1 + \left(\sum_{i=1}^n (x_i^2/4000) - \prod_{i=1}^n (\cos(x_i/\sqrt{i}) + 1) \right) \right)^{-1}$
- $x_i \in [-5.12, 5.12]$
- **Glob. Max.:** $Gr(0, \dots) = 10$

B.11. (R_n) Rosenbrock (R_2 plotted in Fig. 9f)

Note: R_2 and R_5 are used in the paper.

- **Dim.:** n
- $R_n(\vec{x}) = \sum_{i=1}^{n-1} (100 \cdot (x_i^2 - x_{i+1})^2 + (x_i - 1)^2)$
- $x_i \in [-5, 10]$
- **Glob. Min.:** $R_n(1, \dots) = 0$

Remark

We have plotted most of the test functions used in the paper to show what kind of problem they illustrate.

The functions with more than two variables are drawn only for the first two dimensions. The ranges used in the drawings can be different from those used in the tests because of legibility. Finally, the local optima may not appear because of the scale [Figs. 9 and 10].

References

- [1] G. Bilchev, I.C. Parmee, The ant colony metaphor for searching continuous design spaces, *Lect. Notes Comput. Sci.* 993 (1995) 25–39.
- [2] E. Bonabeau, M. Dorigo, G. Theraulaz, *Swarm Intelligence, From Natural to Artificial Systems*, Oxford University Press, Oxford, 1999.
- [3] S. Camazine, J.L. Deneubourg, N. Franks, J. Sneyd, G. Theraulaz, E. Bonabeau, *Self-organization in Biological Systems*, Princeton University Press, Princeton, NJ, 2000.
- [4] R. Chelouah, P. Siarry, A continuous genetic algorithm designed for the global optimization, *J. Heuristics* 6 (2000) 191–213.
- [5] R. Chelouah, P. Siarry, Tabu search applied to global optimization, *Eur. J. Oper. Res.* 123 (2000) 256–270.
- [6] A. Colomi, M. Dorigo, V. Maniezzo, Distributed optimization by ant colonies, in: F. Varela, P. Bourguine (Eds.), *Proceedings of the First European Conference on Artificial Life, ECAL'91*, Elsevier, Paris, France, 1992, pp. 34–142.
- [7] J. Dréo, *Modélisation de la mobilisation chez les fourmis*, Mémoire de DEA, Université Paris7 & Université Libre de Bruxelles, 2001.
- [8] R. Eberhart, J. Kennedy, Y. Shi, *Swarm Intelligence, Evolutionary Computation*, Morgan Kaufmann, Los Altos, CA, 2001.
- [9] D.B. Fogel, *Evolutionary Computation: The Fossil Record*, IEEE Press, 1998, Chapter 7, pp. 481–484.
- [10] F. Glover, M. Laguna, *Tabu Search*, Kluwer Academic Publishers, Dordrecht, 1997.
- [11] C. Hewitt, Viewing control structures as patterns of passing messages, *J. Artif. Intell.* 8 (3) (1977) 323–364.
- [12] J. Holland, Outline for logical theory of adaptive systems, *J. ACM* 9 (3) (1962) 297–314.
- [13] B. Hölldobler, E.O. Wilson, *The Ants*, Springer, Berlin, 1990.
- [14] M. Mathur, S.B. Karale, S. Priye, V.K. Jyaraman, B.D. Kulkarni, Ant colony approach to continuous function optimization, *Ind. Eng. Chem. Res.* 39 (2000) 3814–3822.
- [15] N. Monmarché, G. Venturini, M. Slimane, On how pachycondyla apicalis ants suggest a new search algorithm, *Future Gener. Comput. Syst.* 16 (2000) 937–946.
- [16] R. Storn, K. Price, Differential evolution—a simple and efficient adaptive scheme for global optimization over continuous spaces, Technical Report TR95-012, International Computer Science Institute, Berkeley, CA.
- [17] E.D. Taillard, L.M. Gambardella, M. Gendreau, J.-Y. Potvin, Adaptive memory programming: a unified view of meta-heuristics, *Eur. J. Oper. Res.* 135 (1) (1998) 1–16.
- [18] E. Wilson, B. Hölldobler, Dense heterarchy and mass communication as the basis of organization in ant colonies, *Trend Ecol. Evol.* 3 (1988) 65–68.